Métricas Prometheus — Octopus v1.2.0

O formato comum segue o projeto local **ESP32C6**: identidade pelo MAC STA, reset como gauge 1 com código no label, interfaces de IP identificadas, ocupação da partição do firmware e omissão de integrações indisponíveis.

Funcionamento e configuração web

O Prometheus coleta métricas por **pull**, acessando `http://IP_DO_OCTOPUS:PORTA/metrics`. Não há envio para a API do Prometheus, Pushgateway ou remote-write. O painel continua exclusivamente na porta 80.

1. Abra `http://IP_DO_OCTOPUS/` e a aba **Configurações**.
2. Ative **Ativar endpoint de métricas**.
3. Informe o endereço do servidor Prometheus autorizado: IPv4 ou hostname, sem protocolo, caminho ou porta. Em branco, qualquer cliente pode coletar.
4. Escolha a **porta de métricas no Octopus**, padrão 9120. Valores válidos: 1–65535, exceto 80 (painel) e 3232 (OTA). A porta UDP é independente.
5. Salve. A porta é aplicada sem reiniciar e sem alterar as credenciais Wi-Fi.

As Preferences usam o namespace `prometheus`: `enabled` (bool), `host` (String) e `port` (uint16). Os parâmetros são carregados no boot; a coleta inicia desativada em dispositivos ainda não configurados. Salvar parâmetros inválidos retorna HTTP 400 sem alterar a configuração. Falhas de persistência retornam HTTP 500 e são exibidas no formulário.

Quando há endereço de servidor, o IP de origem precisa coincidir com o IP configurado ou resolvido por DNS. Considere o IP observado pelo Octopus se o Prometheus estiver em container, atrás de NAT ou proxy. Hostnames são resolvidos ao iniciar e a cada 60 segundos. Falhas de DNS bloqueiam a coleta com HTTP 503; clientes de outra origem recebem HTTP 403. O erro 403 mostra `IP recebido`, `IP autorizado` e o servidor configurado; a Serial tambem registra a origem bloqueada. Compare esses enderecos antes de alterar a restricao. Abrir o endpoint em outro computador pelo navegador nao testa a permissao do servidor Prometheus. Para testar a partir do servidor, execute nele `curl http://IP_DO_OCTOPUS:PORTA/metrics`. Em containers/NAT, o endereco observado pode ser diferente do IP esperado. A restrição por IP não é autenticação ou TLS.

O endpoint responde GET com formato Prometheus text 0.0.4, `HELP`, `TYPE`, labels escapados, valores numéricos e quebra de linha final. Não há acesso ao painel ou configuração pela porta de métricas. A página exibe o endpoint, o estado da coleta e um exemplo de configuração com o IP atual do Octopus.

Exemplo no servidor

No `prometheus.yml`, acrescente o job em `scrape_configs` já existente, substituindo IP e porta pelos valores do dispositivo:

```
scrape_configs:
  - job_name: octopus
    scrape_interval: 15s
    scrape_timeout: 5s
```

```
    metrics_path: /metrics
    static_configs:
      - targets: ['192.168.0.98:9120']
```

Recarregue a configuração no servidor e confira o target. Para detectar o dispositivo desligado ou inalcançável, use `up{job="octopus"}`, gerado pelo próprio Prometheus. Um dispositivo desligado não consegue exportar `esp_status_online = 0`; essa métrica indica conexão Wi-Fi enquanto responde.

Identificação e disponibilidade

As métricas abaixo são gauges. Informações textuais ficam em labels; não são valores de amostra textuais.

Todas as amostras, incluindo contadores, GPIOs e arquivos do SD, incluem `device="MAC_DO_OCTOPUS"`. O identificador é o MAC station do dispositivo, no mesmo padrão dos demais ESP32. O MAC é lido da interface STA no boot e permanece estável no modo AP. Em `esp_info`, `model="Octopus"` identifica o modelo; `device` contém o MAC, não o nome.

Exemplo ilustrativo:

```
esp_uptime_seconds{device="AA:BB:CC:DD:EE:FF"} 290.369
esp_last_restart_reason{device="AA:BB:CC:DD:EE:FF",reason="6"} 1
esp_ip_address{device="AA:BB:CC:DD:EE:FF",ip="192.168.0.98",interface="sta"} 1
gpio_digital_output{device="AA:BB:CC:DD:EE:FF",pin="15"} 1
sdcard_log_size_bytes{device="AA:BB:CC:DD:EE:FF",file="/udp_log.txt"} 1024
```

No Grafana/PromQL, filtre por `device` para selecionar o equipamento. Após atualizar o firmware, as séries antigas sem esse label deixam de receber amostras; o histórico anterior permanece no Prometheus.

Métrica	Valor e labels
`esp_info`	1; `device` (MAC), `chip`, `model="Octopus"`
`esp_firmware_version`	1; `version`, `build`
`esp_ip_address`	1; `ip`, `interface="sta"` ou `interface="ap"`; uma série por interface ativa
`esp_mac_address`	1; `mac`
`esp_status_online`	1 com station Wi-Fi conectado, 0 em outro estado
`esp_uptime_seconds`	Segundos desde o boot via temporizador de 64 bits
`esp_last_restart_reason`	1; código numérico ESP-IDF no label `reason`
`esp_boot_count`	Boots acumulados em NVS desde a instalação desta versão
`esp_reboot_count`	Boots acumulados menos o primeiro, incluindo ciclos de energia
`esp_watchdog_resets`	Resets TASK_WDT, INT_WDT e WDT acumulados em NVS

Os contadores persistentes são gravados uma vez por boot no namespace `metric_counts`, chaves `boots` e `watchdogs`. Os valores existentes são preservados e saturam em UINT32_MAX. `esp_reboot_count` é derivado de `boots`; a antiga chave `reboots` deixa de ser usada, sem apagar o histórico de boots. Se a leitura ou gravação dos contadores falhar, suas três séries são omitidas e há aviso na Serial. Erase completo de NVS zera esses valores. `reset_wifi` mantém as métricas e a configuração Prometheus.

Rede, CPU e memória

Métrica	Unidade / significado
`esp_wifi_rssi_dbm`	dBm; omitida sem conexão station
`esp_wifi_channel`	Canal da conexão station; omitida sem conexão
`esp_wifi_disconnect_count`	Eventos station-disconnected desde o boot
`esp_wifi_tx_power`	Potência máxima configurada em dBm; conversão de 0,25 dBm
`esp_cpu_frequency_mhz`	MHz
`esp_cpu_usage_percent`	Percentual ocupado médio dos núcleos, amostrado a cada 5 s
`esp_cpu_temperature_celsius`	Temperatura interna do chip; não é ambiente
`esp_task_count`	Quantidade atual de tarefas FreeRTOS
`esp_free_rtos_tasks`	Alias de `esp_task_count`
`esp_heap_free_bytes`	Heap livre
`esp_heap_total_bytes`	Heap total
`esp_heap_minimum_free_bytes`	Menor heap livre desde o boot
`esp_psram_free_bytes`	PSRAM livre; zero sem PSRAM
`esp_psram_total_bytes`	PSRAM total; zero sem PSRAM
`esp_flash_usage_percent`	Tamanho do sketch / capacidade da partição em execução × 100

Uso de CPU usa os contadores de runtime das tarefas ociosas, sem estimar ocupação pela frequência ou número de tarefas. Sem suporte de runtime stats do FreeRTOS, sem duas amostras completas, com mais de 64 tarefas ou sem snapshot válido, a série é omitida. A leitura usa buffers fixos e deltas por núcleo, incluindo o tratamento de rollover dos contadores. Potência Wi-Fi é omitida sem conexão station ou se o driver falhar; temperatura interna é omitida se a leitura não for finita. Flash é omitida se a partição não for identificada.

Energia, ambiente, relés e GPIO

Como no ESP32C6, estas métricas **não são publicadas** enquanto não existir hardware, calibração e driver configurados no firmware:

Métrica	Unidade / significado previsto
`esp_voltage_v`	Tensão de alimentação em V

Métrica	Unidade / significado previsto
esp_current_ma	Corrente em mA
esp_power_w	Potência em W
esp_battery_percent	Carga da bateria em %
esp_battery_voltage_v	Tensão da bateria em V
sensor_temperature_celsius	Temperatura ambiente em °C
sensor_humidity_percent	Umidade relativa em %
sensor_pressure_hpa	Pressão atmosférica em hPa
sensor_air_quality	Índice do sensor de qualidade do ar
relay_state	Estado lógico do relé (0/1)
relay_activation_count	Ativações acumuladas
relay_runtime_seconds	Tempo ativo acumulado em s
gpio_analog_value	Valor bruto de ADC

Métrica digital	Origem da leitura
gpio_digital_input	GPIO 0; botão de reset, ativo em LOW
gpio_digital_output	GPIOs 2 e 15; LEDs azul e verde

gpio_digital_input{pin="0"} exporta o nível lógico do botão (LOW pressionado). gpio_digital_output exporta os níveis dos LEDs com labels pin="2" e pin="15", além do label comum device. Não são alterados ou amostrados pinos SPI/SD como entradas analógicas.

MQTT

Não há cliente MQTT no Octopus atual. mqtt_connected, mqtt_messages_sent, mqtt_messages_received e mqtt_publish_errors são omitidas até existir integração real, seguindo o ESP32C6.

Aplicação

Métrica	Tipo / significado
application_requests_total	Counter; somente requisições HTTP concluídas desde o boot
application_errors_total	Counter; requisições HTTP concluídas com erro
application_response_time_ms	Gauge; duração da última requisição HTTP concluída, em ms
esp_prometheus_scrapes_total	Counter; coletas autorizadas desde o boot
esp_udp_commands_total	Counter; comandos UDP concluídos desde o boot
esp_udp_errors_total	Counter; erros de comando, falhas TX e descartes por fila cheia
esp_udp_response_time_ms	Gauge; duração do último comando UDP concluído, em ms

As métricas `application_*` seguem o escopo HTTP do ESP32C6, incluindo painel, status, configuração web e coleta. UDP tem séries próprias; um comando com várias respostas conta como um comando. Falhas TX são eventos separados, portanto um comando pode gerar mais de um erro. As durações não medem latência de entrega pela rede. A amostra de um scrape ainda não inclui a duração e o contador de requisições concluídas desse próprio scrape. Não são gravados contadores de tráfego em NVS a cada requisição.

SD card

Métrica	Tipo / significado
`sdcard_mounted`	Gauge; cartão inicializado no boot
`sdcard_metrics_available`	Gauge; 1 com cache de capacidade/ocupação válido, 0 sem dados válidos
`sdcard_capacity_bytes`	Gauge; capacidade física
`sdcard_total_bytes`	Gauge; tamanho do sistema de arquivos
`sdcard_used_bytes`	Gauge; espaço usado
`sdcard_free_bytes`	Gauge; espaço livre
`sdcard_usage_percent`	Gauge; usado / total × 100
`sdcard_log_size_bytes`	Gauge; label `file="/log.txt"` ou `file="/udp_log.txt"`, somente arquivos legíveis existentes
`sdcard_spi_frequency_hz`	Gauge; frequência SPI configurada, 4000000 Hz no Octopus
`sdcard_append_operations_total`	Counter; appends de log concluídos com sucesso
`sdcard_append_errors_total`	Counter; falhas de append de log, incluindo cartão ausente
`sdcard_log_writes_total`	Counter; registros de log gravados com sucesso desde o boot
`sdcard_write_errors_total`	Counter; falhas detectadas de gravação/rotação de logs
`sdcard_log_rotations_total`	Counter; rotações de logs concluídas desde o boot
`sdcard_data_age_seconds`	Gauge; idade do cache válido de capacidade e ocupação
`sdcard_stats_age_seconds`	Gauge; alias de `sdcard_data_age_seconds` mantido para compatibilidade

Sem cartão, montagem e disponibilidade indicam 0; capacidade, ocupação, tamanhos, frequência e idade são omitidas. Um SD montado com estatísticas inválidas mantém `sdcard_metrics_available=0`. Os contadores continuam representando eventos desde o boot. `sdcard_log_writes_total` é equivalente a `sdcard_append_operations_total`; `sdcard_write_errors_total` também inclui falhas de rotação, enquanto `sdcard_append_errors_total` só conta append. O cache de espaço é atualizado a cada 30 segundos pela tarefa de processamento, evitando leitura da FAT durante cada scrape. O firmware não detecta e remonta automaticamente um cartão inserido após o boot. Os contadores de escrita cobrem os logs gerenciados pelo StorageManager, não todas as escritas arbitrárias de `sd_write` ou relatórios de crash. Os tamanhos dos logs são atualizados imediatamente nas escritas de log e revisados junto com o cache de espaço.

Migração e watchdog

O schema comum foi alinhado ao `C:/PC/ESP32/ESP32C6/MetricsService.cpp`. Reset passa a ter valor 1 e código numérico no label `reason`; IP recebe `interface`; GPIO usa apenas `device` e `pin`; caminhos dos logs passam a incluir `/`. Séries antigas permanecem no histórico, mas deixam de receber amostras. O percentual de flash passa a representar a partição, e reboot passa a incluir ciclos de energia. Revise consultas que dependiam dos formatos ou significados antigos. A restrição de acesso ao servidor autorizado permanece.

Foi observado TASK_WATCHDOG em WdtRadioCore0, WdtOtaCore0 e IDLE0. O relatório não identifica sozinho a função que bloqueou; é necessário o ELF correspondente para interpretar os endereços do backtrace. Nesta revisão, o tamanho do firmware é verificado uma vez no boot, em vez de reler a imagem durante cada `/status` ou scrape. A consulta de métricas SD copia um snapshot curto em memória, sem esperar pelo mutex durante operações FAT/SD. O laço de rádio bloqueia por pelo menos um tick, garantindo oportunidade de execução às tarefas de menor prioridade mesmo com outra frequência de tick. O watchdog continua ativo. Essas mudanças precisam ser validadas na placa sob coleta e acesso ao painel.

Referências

- Modelo de coleta do Prometheus
- Formato de exposição
- Configuração de scrape